Reviewer Pack — Free Cumulant Gap in Read-Twice Branching Programs for IP_2

Entry 1d4fc228f6cf · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-07 22:46:26 UTC

Free Cumulant Gap in Read-Twice Branching Programs for IP_2

Entry ID: `1d4fc228f6cf` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-07 22:46:26 UTC

1. Conjecture

Field A (mathematical branch): Free Probability **Field B** (complexity object): Read-Twice Branching Programs

Statement:

> For any read-twice branching program P over n variables, the maximum free cumulant of its path distribution satisfies ρ(P) = O(log n). However, for the inner product mod 2 function IP_2, ρ(IP_2) = Ω(n).

Rationale (proposer's reasoning):

> Free cumulants capture non-commutative independence structures, which may be inherently constrained in read-twice branching programs due to their limited memory. IP_2's high complexity could manifest as unbounded free cumulants, reflecting its resistance to such structured computation.

Taxonomy category: `ACC_LB_via_WILLIAMS` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `33dd17fc3fa0e67e`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    n = 40
    max_instances = 30
    instances_tested = min(max_instances, 2 * n)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            factor = A[i][i]
            for j in range(n):
                A[i][j] /= factor
            for j in range(m):
                if j != i:
                    factor = A[j][i]
                    for k in range(n):
                        A[j][k] -= factor * A[i][k]
        return A

    def matrix_multiply(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[0] * p for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def random_read_twice_bp(n, instances_tested):
        BPs = []
        for _ in range(instances_tested):
            bp = [[random.choice([-1, 1]) for _ in range(2)] for _ in range(n)]
```

```
        BPs.append(bp)
    return BPs

def path_distribution(bp):
    m = len(bp)
    dp = [0] * (1 << m)
    dp[0] = 1
    for i in range(m):
        new_dp = [0] * (1 << m)
        for j in range(1 << m):
            if dp[j] > 0:
                new_dp[j ^ bp[i][0]] += dp[j]
                new_dp[j ^ bp[i][1]] += dp[j]
        dp = new_dp
    return dp

def moments(dp, order):
    moments = [0] * (order + 1)
    for i in range(1 << len(dp)):
        weight = dp[i]
        moment = sum((i & (1 << j)) != 0 for j in range(len(dp))) ** 2
        for k in range(order):
            moments[k] += weight * moment ** (k // 2)
    return moments

def free_cumulants(moments):
    cumulants = [moments[0]]
    for i in range(1, len(moments)):
        cumulant = moments[i]
        for j in range(i):
            cumulant -= sum(cumulants[j] * comb(i, j) * (i - j) ** (i - 2 * j) for j in range(
        cumulants.append(cumulant)
    return cumulants

def IP_2_path_distribution(n):
    A = [[random.choice([-1, 1]) for _ in range(n)] for _ in range(n)]
    dp = [0] * (1 << n)
    dp[0] = 1
    for i in range(n):
        new_dp = [0] * (1 << n)
        for j in range(1 << n):
            if dp[j] > 0:
                new_dp[j ^ A[i][j & (n - 1)]] += dp[j]
        dp = new_dp
    return dp

def IP_2_moments(dp, order):
    moments = [0] * (order + 1)
    for i in range(1 << len(dp)):
        weight = dp[i]
        moment = sum((i & (1 << j)) != 0 for j in range(len(dp))) ** 2
        for k in range(order):
            moments[k] += weight * moment ** (k // 2)
    return moments

def IP_2_free_cumulants(moments):
```

```
        cumulants = [moments[0]]
        for i in range(1, len(moments)):
            cumulant = moments[i]
            for j in range(i):
                cumulant -= sum(cumulants[j] * comb(i, j) * (i - j) ** (i - 2 * j) for j in range(
            cumulants.append(cumulant)
        return cumulants

    BPs = random_read_twice_bp(n, instances_tested)
    IP_2_dp = IP_2_path_distribution(n)
    IP_2_moments = IP_2_moments(IP_2_dp, 40)
    IP_2_cumulants = IP_2_free_cumulants(IP_2_moments)

    max_rho = 5 * math.log(n)
    rho_holds = True
    counterexample = ""

    for bp in BPs:
        dp = path_distribution(bp)
        moments = moments(dp, 40)
        cumulants = free_cumulants(moments)
        if abs(cumulants[-1]) > max_rho:
            rho_holds = False
            counterexample = "BP with high rho found"
            break

    return {
        "metric_name": "rho",
        "metric_value": abs(IP_2_cumulants[-1]),
        "instances_tested": instances_tested,
        "conjecture_holds": rho_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        random.seed(seed)
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rho = sum(r["metric_value"] for r in results) / len(results)
    std_rho = math.sqrt(sum((r["metric_value"] - mean_rho) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rho} std={std_rho} support_fraction={support_fraction
    elif any(not r["conjecture_holds"] for r in results):
        counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
    else:
```

```
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_164f1942.py", line 151, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_164f1942.py", line 119, in run_trial
    IP_2_dp = IP_2_path_distribution(n)
              ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_164f1942.py", line 90, in IP_2_path_dis
    dp = [0] * (1 << n)
         ~~~~^~~~~~~~~~
MemoryError
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed with MemoryError before producing results, preventing validation of conjecture | next: Optimize memory usage for IP_2_path_distribution computation

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	17822
2	preregistration	ollama_remote	qwen3:8b	0	0	20050
3	novelty	ollama_remote	qwen3:8b	0	0	16566
4	novelty	ollama_remote	qwen3:8b	0	0	11014
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15509
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17146
7	judge	ollama_remote	qwen3:8b	0	0	56177

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 154284 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/1d4fc228f6cf.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1d4fc228f6cf.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/1d4fc228f6cf.tar.gz` (if generated)